



Figure 1: The Qt File dialogue

```
stream.seek(0);
QString out_data;
stream >> out_data;
QDebug() << out_data;
```

It's very easy to dissect this code. A file is opened in read-write mode, and its reference passed to the *QTextStream* constructor. The operator '>>' is used to write to the file and '<<' to read from it. A small experiment, if you want: try *QTextStream* for string and byte array data.

Working with *QDataStream* is similar, except that this class deals with raw binary data, not text.

Getting file information

What if we need to work on the file attributes (e.g., the owner, permissions, the path, etc)?

The code given below is self-explanatory:

```
QFileInfo info("./test.txt");
QDebug() << info.absoluteFilePath();
QDebug() << info.baseName();
QDebug() << info.owner();
if(info.permission(QFile::WriteUser)) {
    qDebug() << "Write permission available";
}
```

Buffer

The next important class is *QBuffer*, which provides a *QIODevice*



Figure 2: Adding a resource file



Figure 3: Adding files to prefix

interface to a *QByteArray*. It is useful to emulate a hardware device like a serial port without using real hardware. Just like you get interrupts from the hardware when data is received, here you get a signal when the data arrives in the *QBuffer*.

```
/****** dialog.h *****/
private:
    Ui::Dialog ui;
    QBuffer buf;
    QByteArray ar;
/****** dialog.cpp *****/
Dialog::Dialog(QWidget *parent) :
    QDialog(parent),
    ui(new Ui::Dialog) {
```

TechnoMail - Enterprise Email Server
Anti SPAM, Anti Virus, Email Content Filtering

Firewall, Internet Access Control
Content Filtering, Site Blocking

Bandwidth Management System

Managed Email Hosting Solutions

TechnoInfotech®

1, Vikas Permisses, 11 Bank Street, Fort, Mumbai, India - 400 001. Mobile: 09167399917. info@technoinfotech.com